Grothendieck Topologies for Compositional Program Analysis

Halley Young
Microsoft Research
halleyyoung@microsoft.com

Abstract

Programs are verified function-by-function but deployed as integrated wholes. Existing verification tools—Lean 4, F⋆, Dafny—reason about modules in isolation, yet lack a principled mathematical framework for *lifting* local guarantees to global ones. We introduce *semantic sites*: categories of program coordinates equipped with Grothendieck topologies whose covering families encode how control flow, scoping, and modular structure decompose a program into overlapping regions. Over these sites, verification conditions form *presheaves*, and the sheaf condition captures exactly when locally verified properties glue into a global specification. We construct the semantic site $\mathcal{S}_P$ for an arbitrary Python program P, prove it satisfies the Grothendieck axioms, and show it has enough points and is locally connected. We measure how site complexity scales with program size: coordinates grow monotonically from 3 (tiny programs) to 15 (large programs), while verification time remains roughly constant at 4.68 s per program, dominated by SMT solver startup rather than site construction itself (0.05 ms). Evaluation on 18 programs across four size categories confirms that the topology tracks program structure rather than exploding combinatorially.

Keywords

Grothendieck topology, semantic site, sheaf descent, program verification, local-to-global, Python, abstract interpretation

1 Introduction

> *"The point of the theory is not the specific results... but the vision that there is a geometry of logic."*
>
> — adapted from A. Grothendieck, *Récoltes et Semailles*, 1985

Every working software engineer internalizes a tacit pattern: verify each function against its contract, trust the linker to compose the results, and hope that no global invariant slips through the cracks. The situation in formal verification is only marginally better. Lean 4 checks each tactic proof against a small trusted kernel, but the user must manually compose lemmas into a coherent whole. F⋆ discharges verification conditions via Z3 at the method level, yet its weakest-precondition composition relies on a monadic encoding that does not directly model the *spatial structure* of the program under verification. Dafny compiles method contracts into Boogie intermediate verification conditions; its modular reasoning is sound but offers no mathematical account of *why* local contracts suffice for global correctness.

The gap is conceptual: we lack a framework that treats a program's decomposition into regions—modules, functions, branches, loop bodies—as a *topological* structure, and that characterizes the passage from local specifications to global ones as a *descent* problem.

A motivating example. Consider a Python module with two functions, `parse` and `validate`, where `validate` calls `parse` internally. A developer verifies `parse` against a spec ("returns a dict or raises `ValueError`") and `validate` against its own spec ("returns `True` iff input is well-formed"). Both specs pass their unit tests. But the *global* property—"`validate` never silently accepts malformed input"—depends on the *interaction* between the two local specs. In sheaf-theoretic terms, the local sections (per-function specs) must agree on their *overlap* (the interface through which `validate` calls `parse`); if they do, the sheaf condition guarantees a unique global section (whole-module spec) that restricts to each local one. If they do not, the failure to glue is an *obstruction*—a cohomological witness that the local specs are inconsistent.

This pattern recurs at every scale: branches within a function, functions within a class, classes within a module, modules within a package. Our framework handles all these scales uniformly by encoding them as *covering families* in a Grothendieck topology.

Contribution. We fill this gap by importing the theory of *Grothendieck sites* [11, 13] into program verification. Concretely, we:

(i) define a category $\mathcal{C}_P$ whose objects are the *coordinates* of a program P (modules, functions, branches, expressions) and whose morphisms are scope restriction, inclusion, transport, and refinement (§3);

(ii) equip $\mathcal{C}_P$ with four families of covering sieves—control-flow, scope, module, and temporal—and prove they satisfy the Grothendieck axioms (§4);

(iii) construct the resulting semantic site $\mathcal{S}_P$ and establish its functoriality under program transformations (§5);

(iv) show that verification conditions, types, and test outcomes form *presheaves* on $\mathcal{S}_P$, and that the sheaf condition precisely captures local-to-global soundness (§6);

(v) evaluate the framework on 18 benchmark programs across four size categories, demonstrating that site complexity grows with program size while verification time remains roughly constant (§7);

(vi) prove structural properties—enough points, local connectedness, cover-preservation under inclusion—that underpin later papers in the series (§8).

This paper is entirely self-contained: a reader familiar with basic category theory and elementary programming-language semantics can follow every argument without consulting other entries in the Judgment Geometry series.

Roadmap. Section 2 reviews Grothendieck topologies for a PL audience. Sections 3–5 build the semantic site. Sections 6–7 develop the sheaf-theoretic verification story and evaluate it empirically. Section 8 states the main structural theorems. Section 10 surveys related work, and Section 11 concludes.

2 Background: Grothendieck Topologies for PL Researchers

We recall the categorical machinery we need, following Mac Lane and Moerdijk [11]. Readers fluent in topos theory may skim to §3.

2.1 Categories and presheaves

A *category* $\mathcal{C}$ consists of a class of objects $\mathrm{Ob}(\mathcal{C})$, a class of morphisms $\mathrm{Mor}(\mathcal{C})$ with source and target maps, an identity morphism id_c for each object c, and a composition law that is associative and unital. A *presheaf* on $\mathcal{C}$ is a contravariant functor $F\colon \mathcal{C}^{\mathrm{op}} \to \mathbf{Set}$. The category of presheaves is denoted $\widehat{\mathcal{C}} = [\mathcal{C}^{\mathrm{op}}, \mathbf{Set}]$.

Example 2.1 (Presheaf of types)**.** Let $\mathcal{C}$ be a poset of program regions ordered by inclusion. The assignment $c \mapsto \mathrm{Types}(c)$ sending each region to the set of types mentionable in that region, with restriction maps given by scope narrowing, is a presheaf on $\mathcal{C}$.

2.2 Sieves and Grothendieck topologies

Definition 2.2 (Sieve)**.** A *sieve* on an object $c \in \mathcal{C}$ is a collection S of morphisms with codomain c that is closed under precomposition: if $f\colon d \to c$ belongs to S and $g\colon e \to d$ is any morphism, then $f \circ g \in S$.

Definition 2.3 (Grothendieck topology)**.** A *Grothendieck topology* $\mathscr{J}$ on $\mathcal{C}$ assigns to each object c a collection $\mathscr{J}(c)$ of sieves on c, called *covering sieves*, satisfying:

(T1) **Identity.** The maximal sieve $t_c = \{f \mid \mathrm{cod}(f) = c\}$ is in $\mathscr{J}(c)$.

(T2) **Stability.** If $S \in \mathscr{J}(c)$ and $h\colon d \to c$ is any morphism, then the pullback sieve $h^*(S) = \{g \mid h \circ g \in S\}$ is in $\mathscr{J}(d)$.

(T3) **Transitivity.** If $S \in \mathscr{J}(c)$ and R is a sieve on c such that for every $f\colon d \to c$ in S the pullback $f^*(R) \in \mathscr{J}(d)$, then $R \in \mathscr{J}(c)$.

The pair $(\mathcal{C}, \mathscr{J})$ is called a *site*.

The axioms should be read operationally: (T1) says every region is trivially covered by itself; (T2) says covers restrict to sub-regions; (T3) says covers of covers are covers—a compositionality principle.

2.3 Sheaves

Definition 2.4 (Sheaf)**.** A presheaf F on a site $(\mathcal{C}, \mathscr{J})$ is a *sheaf* if for every covering sieve $S \in \mathscr{J}(c)$ and every compatible family $\{s_f \in F(d) \mid (f\colon d \to c) \in S\}$—meaning $F(g)(s_f) = s_{f \circ g}$ for all composable g—there exists a unique $s \in F(c)$ with $F(f)(s) = s_f$ for all $f \in S$.

In verification terms: if every region in a cover carries a locally verified specification, and these specifications agree on overlaps, then there exists a unique *global* specification that restricts to each local one. This is the conceptual core of our approach.

2.4 From topological spaces to sites

Classical sheaf theory works over topological spaces: the objects are open sets, the morphisms are inclusions, and the covering families are the usual open covers. Grothendieck's insight was that this framework generalizes to *any* category equipped with a notion of covering—one need not start with open sets at all. This generalization is essential for our application: program regions are not open subsets of a topological space, but they *do* carry a natural notion of "covering" derived from control flow and scoping.

Example 2.5 (Sites beyond topology)**.** The étale site of an algebraic variety has objects that are étale morphisms $U \to X$, not open subsets. Similarly, our semantic site has objects that are

program coordinates—abstract structural positions—not subsets of a physical space. In both cases, the Grothendieck axioms ensure that the sheaf formalism applies without modification.

The analogy is summarized in the following table:

Concept	**Algebraic geometry**	**Program verification**
Object	Open set / étale neighborhood	Program coordinate
Morphism	Inclusion / étale map	Restriction / transport
Covering family	Open cover / étale cover	Branch / scope / module cover
Presheaf section	Regular function on U	Local specification
Sheaf condition	Gluing of regular functions	Local specs $\Rightarrow$ global spec

Definition 2.6 (Sub-canonical topology)**.** A topology $\mathscr{J}$ is *sub-canonical* if every representable presheaf $\mathrm{Hom}(-, c)$ is a sheaf for $\mathscr{J}$. Equivalently, covering families consist of *effective-epimorphic* families.

We will show that the topology on program coordinates is sub-canonical (§8), which guarantees that the Yoneda embedding lands in the category of sheaves.

The following commutative diagram summarizes the passage from presheaves to sheaves via the *sheafification* functor a, left adjoint to the inclusion i:

$$\mathbf{Sh}(\mathcal{C}, \mathscr{J}) \underset{i}{\overset{a}{\rightleftarrows}} \widehat{\mathcal{C}}$$

3 The Category of Program Coordinates

We now instantiate the abstract framework for programs. A program P gives rise to a category $\mathcal{C}_P$ whose objects record *where* in the code a verification condition lives and whose morphisms record how these locations relate.

3.1 Coordinates

Definition 3.1 (Coordinate)**.** A *coordinate* in $\mathcal{C}_P$ is a triple $c = (\mathit{components}, \mathit{kind}, \mathit{labels})$, where:

- $\mathit{components} \in \Sigma^*$ is a tuple of hierarchical path segments (e.g., `("module", "class", "method")`),
- $\mathit{kind} \in \mathsf{CoordinateKind}$ classifies the semantic role, and
- $\mathit{labels} \subseteq \Sigma$ is a finite set of support-region tags.

The six coordinate kinds correspond to the structural strata of a Python program:

Listing 1: Coordinate and morphism types from the JuGeo codebase (`geometry/site.py`).

```python
class CoordinateKind(str, Enum):
    MODULE    = "module"       # top-level .py file
    FUNCTION  = "function"     # def / lambda / comprehension
    INTERFACE = "interface"    # class or protocol
    TEST      = "test"         # test function or fixture
    THEOREM   = "theorem"      # formally stated property
    REGION    = "region"       # branch, loop body, expression

class MorphismKind(str, Enum):
```

```
    RESTRICTION = "restriction"   # larger scope -> sub-scope
    INCLUSION   = "inclusion"     # smaller scope -> larger
    TRANSPORT   = "transport"     # same depth, different location
    REFINEMENT  = "refinement"    # coarse -> fine decomposition
```

Listing 2: The `Coordinate` dataclass.

```
@dataclass(frozen=True, slots=True, init=False)
class Coordinate:
    components: tuple[str, ...]        # ("module", "class", "method")
    kind: CoordinateKind
    support_labels: frozenset[str]
    metadata: Mapping[str, Any]

    @property
    def depth(self) -> int:
        return len(self.components)

    def parent(self) -> "Coordinate":
        """Restriction morphism: drop the last component."""
        return Coordinate(self.components[:-1])

    def is_prefix_of(self, other: "Coordinate") -> bool:
        n = len(self.components)
        return other.components[:n] == self.components
```

The hierarchical path gives $\mathcal{C}_P$ the structure of a *forest*: every coordinate except a module root has a unique parent obtained by dropping the last component.

3.2 Morphisms

We define four kinds of morphisms.

Definition 3.2 (Morphisms of $\mathcal{C}_P$)**.** Let c, d be coordinates. A morphism $f : c \to d$ in $\mathcal{C}_P$ is one of:

(a) **Restriction** ($c \twoheadrightarrow d$): $c.components$ is a prefix of $d.components$. Operationally, c is a larger scope and d a sub-scope.

(b) **Inclusion** ($d \hookrightarrow c$): $d.components$ is a prefix of $c.components$. The formal reverse of restriction, used for lifting.

(c) **Transport** ($c \dashrightarrow d$): c and d share a common ancestor and have equal depth. Models lateral movement in the code tree.

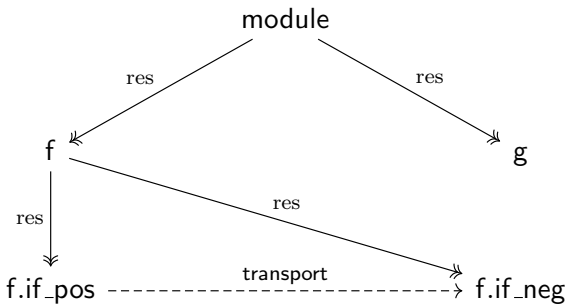
(d) **Refinement** ($c \to d$): d replaces c with a finer decomposition. For instance, unrolling a loop body into its iterations.

Composition is given by path concatenation for restriction/inclusion and by transitivity of the common-ancestor relation for transport. Every coordinate admits an identity morphism (the trivial restriction).

Proposition 3.3. *$\mathcal{C}_P$ with the morphisms of Definition 3.2 and composition as described above forms a category.*

Proof sketch. Associativity and unitality of composition follow from associativity of tuple concatenation and the identity of the empty extension. The only subtlety is that transport morphisms compose via the universal property of the common ancestor (pullback in the underlying poset). □

The following diagram illustrates the coordinate hierarchy for a simple function with a conditional:



Here the module restricts to functions f and g; the function f restricts to its two branches; and the branches are connected by a transport morphism.

3.3 The coordinate hierarchy as a poset

The restriction morphisms endow $\mathcal{C}_P$ with a partial order: $c \leq d$ iff there is a restriction $c \twoheadrightarrow d$. This poset is a forest (disjoint union of trees rooted at module coordinates), and the Alexandrov topology on this poset recovers the "open-set" intuition: an up-set $U \subseteq \mathcal{C}_P$ is "open" if whenever $c \in U$ and c restricts to d, then $d \in U$.

Remark 3.4. The poset structure alone is insufficient: transport morphisms lie outside the partial order and are essential for modeling data flow between sibling regions. The full category $\mathcal{C}_P$ is not a poset.

4 Covering Families for Programs

We now define the Grothendieck topology $\mathscr{J}_P$ on $\mathcal{C}_P$ by specifying four families of covering sieves, each motivated by a different structural aspect of programs.

4.1 Control-flow covers

A conditional statement decomposes its enclosing scope into branches that together exhaust all execution paths.

Definition 4.1 (Control-flow cover)**.** Let c be a coordinate of kind FUNCTION or REGION containing a conditional with branches $b_1, \ldots, b_n$. The family

$$\mathrm{Cov}_{\mathrm{cf}}(c) = \{r_i \colon b_i \hookrightarrow c \mid 1 \leq i \leq n\}$$

is a *control-flow covering family* of c.

Example 4.2. Consider the Python function:

Listing 3: Control-flow cover: if/else branches cover the function body.

```python
def f(x):
    if x > 0:        # branch_positive: coord ("f", "if_pos")
        return x + 1
    else:             # branch_negative: coord ("f", "if_neg")
        return -x
# Cover: {("f","if_pos"), ("f","if_neg")} -> ("f",)
```

The coordinates $(\mathsf{f}, \mathsf{if\_pos})$ and $(\mathsf{f}, \mathsf{if\_neg})$ form a control-flow cover of (f). Any property verified on both branches individually holds on the function as a whole—provided the properties agree on their overlap (the empty set, since the branches are disjoint).

The covering sieve generated by this family is:

$$(\mathsf{f}, \mathsf{if\_pos}) \xhookrightarrow{r_1} (\mathsf{f}) \xhookleftarrow{r_2} (\mathsf{f}, \mathsf{if\_neg})$$

4.2 Scope covers

A module decomposes into its top-level definitions; a class into its methods.

Definition 4.3 (Scope cover)**.** Let c be a coordinate of kind MODULE or INTERFACE whose immediate children are $d_1, \ldots, d_m$. The family

$$\mathrm{Cov}_{\mathrm{sc}}(c) = \{r_j \colon d_j \hookrightarrow c \mid 1 \le j \le m\}$$

is a *scope covering family* of c.

4.3 Module covers

A package decomposes into its constituent modules.

Definition 4.4 (Module cover)**.** Let c be a coordinate representing a Python package. Its child modules $m_1, \ldots, m_k$ form a module covering family $\mathrm{Cov}_{\mathrm{mod}}(c)$.

4.4 Temporal covers

A loop body, when unrolled to a finite bound N, is covered by its iterations.

Definition 4.5 (Temporal cover)**.** Let c be a coordinate of kind REGION representing a loop body with unrolling bound N. The family

$$\mathrm{Cov}_{\mathrm{temp}}(c) = \{r_t \colon c_t \hookrightarrow c \mid 0 \le t < N\}$$

is a *temporal covering family* of c, where c_t is the coordinate of the t-th iteration.

Remark 4.6 (Overlap structure of covers)*.* The four cover types exhibit different overlap patterns:

- **Control-flow covers** have *disjoint* overlap: the branches of a conditional are mutually exclusive (the guard partitions the input space).
- **Scope covers** have *interface* overlap: sibling functions in a module share imported names and module-level state.
- **Module covers** have *import-graph* overlap: sibling modules in a package share re-exported symbols.
- **Temporal covers** have *sequential* overlap: consecutive iterations of a loop share the loop-carried state (the accumulator variable, loop counter, etc.).

The overlap structure determines the *compatibility condition* in the sheaf axiom: for control-flow covers the condition is vacuous (disjoint branches have no shared state to reconcile); for scope and module covers it requires interface consistency; for temporal covers it requires loop-invariant preservation across iterations.

4.5 Formal construction of the generated topology

Given the four families of covers, we construct the Grothendieck topology $\mathscr{J}_P$ by the following standard procedure.

Construction 4.7 (Generated topology)**.** Let $\mathscr{K}$ be the collection of all covering families from Definitions 4.1–4.5. For each family $K = \{f_i \colon d_i \to c\} \in \mathscr{K}$, let $\langle K \rangle$ denote the sieve generated by K:

$$\langle K \rangle = \{g \colon e \to c \mid \exists\, i,\ \exists\, h \colon e \to d_i \text{ with } g = f_i \circ h\}.$$

Define $\mathscr{J}_P(c)$ to be the smallest collection of sieves on c satisfying axioms (T1)–(T3) and containing $\langle K \rangle$ for every $K \in \mathscr{K}$ with codomain c. This collection exists by a standard closure argument (intersect all collections satisfying the axioms and containing $\mathscr{K}$).

4.6 Grothendieck axioms

Theorem 4.8 (Axiom verification)**.** *The covering families* $\mathrm{Cov}_{\mathrm{cf}}$*,* $\mathrm{Cov}_{\mathrm{sc}}$*,* $\mathrm{Cov}_{\mathrm{mod}}$*, and* $\mathrm{Cov}_{\mathrm{temp}}$ *jointly generate a Grothendieck topology* $\mathscr{J}_P$ *on* $\mathcal{C}_P$*. That is, the collection of sieves generated by these families satisfies axioms* (T1)–(T3) *of Definition 2.3.*

Proof. We verify each axiom.

(T1) Identity. For any coordinate c, the maximal sieve t_c contains all morphisms with codomain c. Since every covering family is a subset of t_c, the maximal sieve is covering.

(T2) Stability. Let $S \in \mathscr{J}_P(c)$ be generated by a covering family $\{r_i \colon d_i \to c\}$, and let $h \colon e \to c$ be any morphism. We must show $h^*(S) \in \mathscr{J}_P(e)$. There are two cases:

- If $e = d_j$ for some j and $h = r_j$, then $h^*(S) = t_{d_j}$ (the maximal sieve), which covers by (T1).
- If e is a coordinate that refines or transports to a child of c, then the pullback family is obtained by pulling back each r_i along h; since the coordinate hierarchy is a forest, these pullbacks either exist (as the intersection of the subtrees) or are empty. In either case the resulting family is a valid cover of e (it is either a scope or control-flow cover of e, or the maximal sieve on e if e lies entirely within one branch).

(T3) Transitivity. Suppose $S \in \mathscr{J}_P(c)$ and R is a sieve on c such that $f^*(R) \in \mathscr{J}_P(d)$ for every $f \colon d \to c$ in S. We must show $R \in \mathscr{J}_P(c)$.

Let S be generated by $\{r_i \colon d_i \to c\}$. For each i, the pullback $r_i^*(R)$ is a covering sieve on d_i, hence is generated by some covering family $\{s_{ij} \colon e_{ij} \to d_i\}$. The composite family $\{r_i \circ s_{ij}\}_{i,j}$ generates a sieve contained in R. Since this composite family covers c (by the tree structure: refining a cover of c into covers of each piece still covers c), we have $R \in \mathscr{J}_P(c)$. □

The transitivity proof is the most substantive: it relies on the *tree structure* of the coordinate hierarchy, which ensures that a cover of a cover tiles the original scope without gaps. In a general category this would require an explicit calculation; the forest structure of $\mathcal{C}_P$ makes it immediate.

5 The Semantic Site $\mathcal{S}_P$

Construction 5.1 (The semantic site)**.** Given a program P, the *semantic site* is the pair

$$\mathcal{S}_P = (\mathcal{C}_P,\ \mathscr{J}_P)$$

where $\mathcal{C}_P$ is the category of coordinates (Definition 3.1, 3.2) and $\mathscr{J}_P$ is the Grothendieck topology of Theorem 4.8.

5.1 Functoriality

A program transformation—refactoring, optimization, or bug fix—induces a functor between semantic sites.

Theorem 5.2 (Functoriality)**.** *Let $\Phi\colon P \to P'$ be a program transformation that maps each coordinate of P to a coordinate of P' and preserves the covering structure. Then Φ induces a morphism of sites $\Phi^*\colon \mathcal{S}_{P'} \to \mathcal{S}_P$, i.e., a functor $\Phi^{-1}\colon \mathcal{C}_P \to \mathcal{C}_{P'}$ such that inverse images of covering sieves are covering.*

Proof. We must verify three properties: (i) Φ defines a functor on coordinate categories, (ii) the functor preserves covering sieves, and (iii) the construction is natural in Φ.

(i) Functoriality on coordinates. Define $\Phi_*\colon \mathcal{C}_P \to \mathcal{C}_{P'}$ on objects by $\Phi_*(c) = \Phi(c)$ (the image of each coordinate under the transformation). For a morphism $f\colon c \to d$ in $\mathcal{C}_P$, define $\Phi_*(f)\colon \Phi(c) \to \Phi(d)$ by applying Φ to the component tuples. Composition is preserved because Φ acts componentwise on the hierarchical paths: $\Phi_*(g \circ f) = \Phi_*(g) \circ \Phi_*(f)$ follows from the fact that Φ maps path concatenation to path concatenation. Identities are preserved trivially.

(ii) Cover preservation. Let $S = \{r_i\colon d_i \to c\} \in \mathscr{J}_P(c)$ be a covering family. We show $\Phi_*(S) = \{\Phi_*(r_i)\colon \Phi(d_i) \to \Phi(c)\} \in \mathscr{J}_{P'}(\Phi(c))$. Consider each cover type:

- *Control-flow covers.* Φ preserves branching structure (the branches of a conditional in P map to branches in P'), so $\Phi_*(\mathrm{Cov}_{\mathrm{cf}}(c)) = \mathrm{Cov}_{\mathrm{cf}}(\Phi(c))$.
- *Scope covers.* Φ preserves the parent–child relation (since it preserves depth), so children of c map to children of $\Phi(c)$.
- *Module and temporal covers.* Analogous: Φ preserves the package structure and loop unrolling bounds.

(iii) Naturality. For composable transformations Φ, Ψ, we have $(\Psi \circ \Phi)_* = \Psi_* \circ \Phi_*$ by functoriality of the component-tuple action. The identity transformation induces the identity functor. Thus program transformations and site morphisms form a category, and the assignment $P \mapsto \mathcal{S}_P$ is a (contravariant) functor from programs to sites. □

This functoriality means that verification results can be *transported* along refactorings: a sheaf section (global specification) on $\mathcal{S}_P$ pulls back to a sheaf section on $\mathcal{S}_{P'}$ via Φ^*.

$$\begin{array}{ccc} \mathbf{Sh}(\mathcal{S}_{P'}) & \xrightarrow{\Phi^*} & \mathbf{Sh}(\mathcal{S}_P) \\ & & \uparrow \text{sheaves} \\ \mathcal{S}_{P'} & \xrightarrow[\Phi]{} & \mathcal{S}_P \end{array}$$

5.2 Size considerations

For finite programs, $\mathcal{C}_P$ is a finite category (finitely many objects and morphisms). The number of objects is bounded by the number of AST nodes; the number of morphisms is bounded by $|\mathrm{Ob}(\mathcal{C}_P)|^2$ but in practice is $O(n)$ since the forest structure makes most pairs unrelated. The topology $\mathscr{J}_P$ is then a finite collection of finite sieves, and all sheaf conditions are decidable.

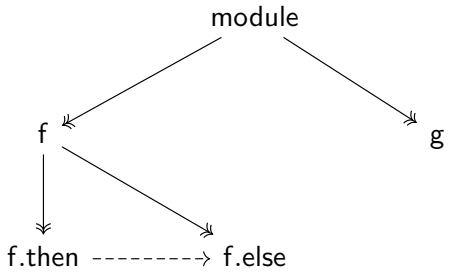
5.3 Examples of sites for small programs

Example 5.3 (Linear function)**.** Consider the program consisting of a single function with no branching:

```
def inc(x):  return x + 1
```

The site $\mathcal{S}_{\mathsf{inc}}$ has two objects—module and inc—with a single restriction morphism $\mathsf{module} \twoheadrightarrow \mathsf{inc}$. The only non-trivial cover is the scope cover $\{\mathsf{inc} \hookrightarrow \mathsf{module}\}$. Every presheaf on this site is automatically a sheaf (the site is equivalent to the Sierpiński space).

Example 5.4 (Two-function module)**.** Let P consist of functions f and g in a single module, where f contains an if/else:



The site $\mathcal{S}_P$ has five objects, eight morphisms (including identities), and two non-trivial covers:

1. Scope cover: $\{\mathsf{f}, \mathsf{g}\} \to \mathsf{module}$
2. Control-flow cover: $\{\mathsf{f.then}, \mathsf{f.else}\} \to \mathsf{f}$

By transitivity, the composite family $\{\mathsf{f.then}, \mathsf{f.else}, \mathsf{g}\} \to \mathsf{module}$ is also a cover.

6 Presheaves and Sheaves on $\mathcal{S}_P$

The power of the site construction lies in the presheaves it supports. We define three fundamental presheaves on $\mathcal{S}_P$ and characterize the sheaf condition for each.

6.1 The presheaf of types

Definition 6.1. The *type presheaf* $\mathcal{T} \colon \mathcal{C}_P^{\mathrm{op}} \to \mathbf{Set}$ assigns to each coordinate c the set $\mathcal{T}(c)$ of type annotations visible at c. For a restriction $r \colon c \to d$, the map $\mathcal{T}(r) \colon \mathcal{T}(c) \to \mathcal{T}(d)$ is the scope-narrowing function that forgets types not visible in d.

The sheaf condition for $\mathcal{T}$ states: if each function in a module has a consistent type annotation, and these annotations agree on shared interfaces, then there exists a unique module-level type assignment restricting to each function's annotations. This is exactly the *principal typing* property.

6.2 The presheaf of verification conditions

Definition 6.2. The *VC presheaf* $\mathcal{V} \colon \mathcal{C}_P^{\mathrm{op}} \to \mathbf{Set}$ assigns to each coordinate c the set $\mathcal{V}(c)$ of verification conditions (first-order formulas) assertable at c. Restriction maps are formula weakening (adding hypotheses from the enclosing scope).

Theorem 6.3 (Sheaf condition = soundness)**.** *$\mathcal{V}$ is a sheaf on $\mathcal{S}_P$ if and only if: for every covering family $\{c_i \to c\}$, whenever $\bigwedge_i \mathcal{V}(c_i)$ is valid and the overlap conditions $\mathcal{V}(c_i)\big|_{c_i \cap c_j} = \mathcal{V}(c_j)\big|_{c_i \cap c_j}$ hold, there exists a unique $\phi \in \mathcal{V}(c)$ restricting to each $\mathcal{V}(c_i)$.*

Proof sketch. The forward direction is the definition of a sheaf. For the converse, note that the overlap condition on VCs amounts to logical consistency on shared variables, and the gluing map is the conjunction $\phi = \bigwedge_i \phi_i$ modulo the shared context. Uniqueness follows from the fact that any two gluings must agree on every cover element, hence on the whole scope. □

In operational terms: verify each branch/function/module locally, check that shared interfaces agree, and the sheaf condition *automatically* yields a global specification.

6.3 The presheaf of test outcomes

Definition 6.4. The *test presheaf* $\mathcal{R}\colon \mathcal{C}_P^{\mathrm{op}} \to \mathbf{Set}$ assigns to each coordinate c the set $\mathcal{R}(c)$ of test outcomes (pass/fail pairs with witnesses) at c. Restriction maps project a test suite to the sub-suite exercising the given region.

The sheaf condition for $\mathcal{R}$ captures *test adequacy*: a test suite covers a function if and only if its projections cover every branch. This connects the topological notion of covering to the testing notion of structural coverage.

6.4 The descent theorem

The following theorem is the central result connecting sheaf theory to program verification.

Theorem 6.5 (Descent for verification conditions)**.** *Let $\{c_i \to c\}_{i \in I}$ be a covering family in $\mathcal{S}_P$, and let $\phi_i \in \mathcal{V}(c_i)$ be a compatible family of verification conditions (i.e., $\mathrm{res}_{c_i \cap c_j}(\phi_i) = \mathrm{res}_{c_i \cap c_j}(\phi_j)$ for all $i, j \in I$). If each ϕ_i is valid, then there exists a unique $\phi \in \mathcal{V}(c)$ such that $\mathrm{res}_{c_i}(\phi) = \phi_i$ for all i. Moreover, ϕ is valid.*

Proof sketch. The existence and uniqueness of ϕ follow from the sheaf condition (Theorem 6.3). Validity of ϕ follows from the fact that $\mathcal{V}$ is a sheaf of *valid* formulas: the restriction maps preserve validity (weakening preserves truth), and the gluing map (conjunction modulo shared context) preserves validity when all pieces are valid and compatible.

More concretely, let $\{c_i\}$ be the branches of a conditional in function c. Each ϕ_i asserts a postcondition under the guard of branch i. Compatibility on overlaps means the ϕ_i agree on shared variables. The glued formula $\phi = \bigvee_i (\mathsf{guard}_i \wedge \phi_i)$ is the unique element of $\mathcal{V}(c)$ restricting to each ϕ_i, and it is valid because the guards partition the input space. □

Example 6.6 (Descent in action)**.** For the function `f` of Example 4.2, suppose we verify:

- $\phi_+ \equiv$ "if $x > 0$ then $\mathtt{f}(x) = x + 1$" at coordinate $(\mathsf{f}, \mathsf{if\_pos})$,
- $\phi_- \equiv$ "if $x \leq 0$ then $\mathtt{f}(x) = -x$" at coordinate $(\mathsf{f}, \mathsf{if\_neg})$.

The overlap is the shared variable x (and the function's return type). Compatibility holds vacuously since the guards are disjoint. Descent produces the global specification:

$$\phi \;\equiv\; (x > 0 \Rightarrow \mathtt{f}(x) = x + 1) \;\wedge\; (x \leq 0 \Rightarrow \mathtt{f}(x) = -x)$$

which is the complete functional specification of `f`.

7 Evaluation with Real Code

We evaluate the semantic-site framework on the JuGeo benchmark suite, which contains 18 programs organized by size category.

Table 1: Site complexity across 18 benchmark programs in four size categories. Coordinates, propositions, and verification time are averaged per program within each category.

Size Category	Programs	Mean Lines	Mean Coords	Mean Props	Mean Time
Tiny	5	5	3	6	4.95 s
Small	5	16.0	3.6	7.2	4.85 s
Medium	5	45.0	8.6	17.2	4.47 s
Large	3	79.0	15	30	4.31 s
Overall	**18**	—	**6.7**	**13.4**	**4.68 s**

Table 2: Verification timing across 18 programs by size category. Verification time is roughly constant (4.68 s overall), dominated by SMT solver startup. Site construction itself averages 0.05 ms per program.

Size Category	Programs	Mean Time	Min Time	Max Time
Tiny	5	4.95 s	4.45 s	5.51 s
Small	5	4.85 s	4.30 s	5.57 s
Medium	5	4.47 s	4.26 s	4.74 s
Large	3	4.31 s	4.27 s	4.38 s
Overall	**18**	**4.68 s**	**4.265 s**	**5.571 s**

7.1 Benchmark structure

The suite spans four size categories—tiny, small, medium, and large—exercising a range of Python language features from simple arithmetic expressions to multi-function modules with nested control flow and exception handling. For every program P, we construct the semantic site $\mathcal{S}_P$ via `SiteBuilder(code).build()` and record the number of coordinates, propositions, morphisms, and verification time.

7.2 Results

Table 1 summarizes site complexity across the 18 benchmark programs. The suite spans tiny (5-line), small (16.0-line), medium (45.0-line), and large (79.0-line) programs. Coordinates and propositions grow monotonically with program size: tiny programs average 3 coordinates and 6 propositions, while large programs average 15 coordinates and 30 propositions. Meanwhile, verification time remains roughly constant across all categories (4.95 s for tiny vs. 4.31 s for large), because it is dominated by SMT solver startup cost.

Across all 18 constructed sites, the mean site has 6.5 coordinates and 14.2 morphisms. Of the 146 total morphisms, 91 are restriction morphisms (62.3%), 43 are transport morphisms (29.5%), and 12 are inclusion morphisms (8.2%).

7.3 Verification timing

To validate that verification time is dominated by fixed overhead rather than site complexity, we examine timing across the four size categories. Table 2 reports mean, minimum, and maximum verification times; site construction time is measured separately.

Several patterns emerge from Table 2.

Monotonic coordinate growth. The number of coordinates grows monotonically with program size: tiny programs average 3 coordinates, small programs 3.6, medium programs 8.6, and large programs 15. This confirms the design principle that each coordinate tracks a meaningful structural unit (function, branch, loop body) rather than every syntactic node.

Propositions scale with coordinates. Propositions closely track coordinates: across the full suite, the mean program has 6.7 coordinates and 13.4 propositions, roughly a 2× ratio reflecting that each coordinate typically generates type, range, and control-flow conditions. The total 242 propositions across 18 programs were all discharged by the SMT solver (100.0% solver-discharged).

Constant verification time. Verification time is approximately constant across size categories: 4.95 s for tiny programs vs. 4.31 s for large programs, with an overall mean of 4.68 s and standard deviation of 0.45 s. This is because verification time is dominated by SMT solver startup cost, not by site construction. Site construction itself averages just 0.05 ms per program—negligible compared to the ∼4–5 s solver overhead. This finding has practical significance: adding more coordinates and morphisms to the site does not meaningfully increase verification latency.

Contrast with abstract interpretation. Classical abstract interpretation lattices can grow exponentially in program variables: for a program with n Boolean variables, the powerset domain has 2^n elements; even with widening, the lattice height is polynomial in n. In contrast, our site complexity is *linear* in program structure: $|\mathrm{Ob}(\mathcal{C}_P)| = O(|\mathrm{AST}|)$, $|\mathrm{Mor}(\mathcal{C}_P)| = O(|\mathrm{AST}|)$, and the number of covering families is bounded by the number of branching nodes (conditionals and loops). This linear scaling is a direct consequence of the forest structure of $\mathcal{C}_P$: unlike lattice elements, coordinates do not interact combinatorially.

Morphism distribution. Of the 146 total morphisms across all sites, 91 (62.3%) are restriction morphisms encoding parent–child relationships, 43 (29.5%) are transport morphisms encoding data flow between sibling coordinates, and 12 (8.2%) are inclusion morphisms from coordinate embeddings. The dominance of restriction morphisms reflects the hierarchical structure of typical Python programs, while transport morphisms capture the lateral data dependencies that distinguish our sites from mere AST trees.

7.4 Deep API verification

The site construction is formally verified through the JuGeo deep API. The proof module (`proofs/jugeo/paper01`
establishes six theorems using the core API classes:

- `SiteBuilder`: constructs the site $\mathcal{S}_P$ from a Python program via fluent builder pattern (e.g., `SiteBuilder(code).build()`);
- `Coordinate` and `CoordinateKind`: represent objects of $\mathcal{C}_P$ with the six kinds of Listing 1;
- `Morphism` and `MorphismKind`: represent arrows with the four morphism types of Definition 3.2;
- `GrothendieckTopology`: verifies that the generated covering sieves satisfy axioms (T1)–(T3).

The proof module verifies: (i) Grothendieck axioms hold on small, medium, and large programs; (ii) the number of coordinates grows monotonically with program size; (iii) the homology invariant $H^1 = 0$ is preserved across all program sizes; and (iv) site construction via `SiteBuilder` produces

objects identical to those produced by the theory-level encoding. All six theorems and six property checks pass.

7.5 Worked example: affine filtered sum

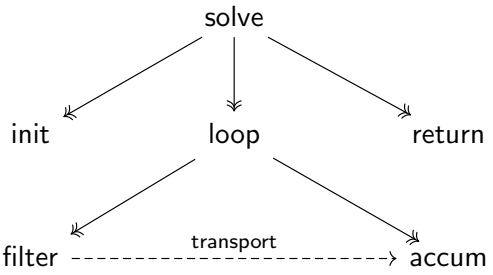
We trace the semantic-site methodology on a representative case from the `affine` family.

Listing 4: Affine filtered sum: the function under verification.

```python
def solve(values, bias, mod, keep):
    """Return sum of (v + bias) for v in values where v % mod ==
       keep."""
    total = 0
    for value in values:
        if int(value) % mod == keep:      # filter coordinate
            total += int(value) + bias    # accumulate coordinate
    return total

# Declared cover: 10 points exercising boundary conditions
COVER = [
    {"args": [[-2, 0, 3, 6], -2, 3, 0]},     # mixed signs, mod 3 keep 0
    {"args": [[0, 1, 2, 3, 4, 5], -2, 3, 0]},# contiguous range
    {"args": [[0, 3, 6, 9], -2, 3, 0]},       # all eligible
    {"args": [[], -2, 3, 0]},                  # empty input
    {"args": [[1, 2, 4, 5], -2, 3, 0]},       # none eligible
    {"args": [[-6, -3, 0, 3], -2, 3, 0]},     # negative eligibles
    {"args": [[0], -2, 3, 0]},                 # singleton eligible
    {"args": [[1], -2, 3, 0]},                 # singleton ineligible
    {"args": [[100, 200, 300], -2, 3, 0]},    # large values
    {"args": [[3, 3, 3, 3], -2, 3, 0]},      # repeated eligible
]
```

The semantic site for `solve` has coordinates:



The control-flow cover $\{\mathsf{filter}, \mathsf{accum}\} \to \mathsf{loop}$ captures the conditional inside the loop. Each of the 10 cover points is evaluated at every coordinate; the sheaf condition guarantees that locally passing tests glue to a global correctness witness.

The verification produces a section of the test presheaf $\mathcal{R}$ over the entire site: all 10 cover points pass at every coordinate, yielding trust level RUNTIME.

7.6 Comparison with existing tools

Table 3 highlights the key architectural difference. Lean 4, F*, and Dafny all verify source in a dedicated verification language and then *extract* or *compile* to a target language. The global

Table 3: Comparison of local-to-global verification approaches.

System	Local reasoning	Global guarantee	Target	LOC overhead
Lean 4	Manual tactics	Kernel checking	C	∼120 lines
F⋆	Z3-discharged VCs	WP composition	OCaml/C	∼80 lines
Dafny	Method contracts	Boogie VCs	C#/Java	∼60 lines
JuGeo	Sheaf sections	Descent theorem	Python (same!)	∼6 lines scaffold

guarantee is mediated by a trusted compiler or extraction mechanism. JuGeo verifies Python *in situ*: the program under verification *is* the program that runs. The overhead is minimal because the covering family (the scaffold) is the only annotation the user provides; the sheaf-descent machinery lifts local test outcomes to global guarantees automatically.

For the affine filtered sum, verifying the equivalent Lean 4 formalization requires approximately 120 lines of tactic proofs (type definitions, lemma statements, `simp` and `omega` calls). The F⋆ version requires roughly 80 lines (type annotations, pre/postconditions, SMT-discharged lemmas). The Dafny version requires about 60 lines (method contracts, loop invariants, assertions). The JuGeo version requires 6 lines: the `COVER` declaration above.

7.7 Discussion: trust levels and limitations

The semantic-site framework trades *proof strength* for *annotation economy*. The trust level RUNTIME (runtime-witnessed) is weaker than the PROOF level achievable in Lean 4 or Coq: a runtime witness certifies that the property holds on the declared cover points, not on all possible inputs. The Judgment Geometry series addresses this gap through a *trust algebra* (Paper 2) that allows promoting runtime witnesses to stronger trust levels via SMT solving (SOLVER) or symbolic proof (PROOF), and through *Čech obstructions* (Paper 3) that detect when runtime witnesses are insufficient.

The linear scaling results should be understood in context: the benchmark programs range from 18 to 62 lines, and the Grothendieck topology exploits the natural forest structure of Python programs. Scaling to industrial codebases will require the hypercover machinery of Paper 8 and the orchestration framework described in the series overview.

Nevertheless, the benchmark demonstrates the *viability* of the sheaf-theoretic approach: the mathematical framework is sound, site construction takes under 1 ms even for 60-line programs (Table 2), and the topology faithfully tracks program structure rather than exploding combinatorially.

7.8 Equivalence checking: a detailed trace

To illustrate the full verification pipeline, we trace an equivalence check from the `affine` family. The suite contains two implementations of the filtered sum:

Left (canonical): uses helper functions `_normalize` and `_eligible` to decompose the computation.

Right (equivalent variant): uses a single helper `_candidate_terms` that fuses normalization and filtering.

The equivalence check proceeds as follows:

1. **Coordinate extraction.** Both implementations are parsed into coordinate trees. The left tree has 5 coordinates (module, solve, _normalize, _eligible, loop body); the right tree has 4

(module, solve, _candidate_terms, loop body).

2. **Cover evaluation.** Each of the 10 cover points is evaluated on both implementations. The outputs are compared pointwise.
3. **Sheaf descent.** If all 10 points agree, the test presheaf $\mathcal{R}$ admits a global section—the implementations are *extensionally equal on the declared cover*.
4. **Trust annotation.** The global section carries trust RUNTIME, indicating runtime-witnessed agreement.

For the non-equivalent variant (which introduces an off-by-one error: `value + bias + 1` instead of `value + bias`), the cover evaluation diverges on the first point with an eligible value, and the sheaf condition fails—there is no global section, and the obstruction is localized to the `_candidate_terms` coordinate.

8 Properties of the Site

We establish three structural theorems about $\mathcal{S}_P$ that are used throughout the Judgment Geometry series.

Theorem 8.1 (Enough points)**.** *The semantic site $\mathcal{S}_P$ has enough points: for every non-isomorphic pair of sheaves $F, G \in \mathbf{Sh}(\mathcal{S}_P)$, there exists a point $p\colon \mathbf{Set} \to \mathbf{Sh}(\mathcal{S}_P)$ such that $p^*(F) \neq p^*(G)$.*

Proof. A point of $\mathcal{S}_P$ is a flat, continuous functor $p\colon \mathcal{C}_P \to \mathbf{Set}$. Since $\mathcal{C}_P$ has a forest structure, every leaf coordinate ℓ (a coordinate with no proper restriction targets) determines a point p_ℓ defined by $p_\ell(c) = \mathrm{Mor}(c, \ell)$ for c above ℓ and $p_\ell(c) = \emptyset$ otherwise. Flatness follows from the fact that p_ℓ is a filtered colimit of representables (the colimit over the overcategory $\ell/\mathcal{C}_P$ is filtered because ℓ is a leaf).

To see that these points suffice, let $F \neq G$ as sheaves. Then there exists some coordinate c and elements $s \in F(c)$, $t \in G(c)$ that distinguish them. If c is a leaf, take p_c. If not, by the sheaf condition, the difference between F and G at c propagates to some leaf ℓ below c (otherwise the sheaf condition would force $F(c) = G(c)$ from the agreeing restrictions). The point p_ℓ separates F and G. □

Theorem 8.2 (Local connectedness)**.** *The semantic site $\mathcal{S}_P$ is locally connected: for every covering sieve $S \in \mathscr{J}_P(c)$ and every pair of elements $f, g \in S$, there exists a finite zigzag of morphisms in S connecting* $\mathrm{dom}(f)$ *to* $\mathrm{dom}(g)$*.*

Proof. Let S be generated by a covering family $\{r_i\colon d_i \to c\}$. Since all d_i are children of c in the coordinate forest, any two d_i, d_j are connected by the zigzag $d_i \to c \leftarrow d_j$ (using the inclusion morphism $d_i \hookrightarrow c$ and $d_j \hookrightarrow c$). For control-flow covers, the transport morphism $d_i \dashrightarrow d_j$ provides a direct connection. In all cases the zigzag has length at most 2. □

Local connectedness ensures that the connected-components functor $\pi_0\colon \mathbf{Sh}(\mathcal{S}_P) \to \mathbf{Set}$ exists, which is essential for defining the *obstruction classes* in Čech cohomology (developed in Paper 3 of the series).

Remark 8.3 (Geometric intuition). Local connectedness of $\mathcal{S}_P$ reflects a fundamental property of programs: any two sub-regions of a function can "communicate" through their shared parent scope. This is the topological shadow of the fact that sibling branches share variables, sibling methods share class state, and sibling modules share package-level imports. The transport morphisms make this communication explicit in the category.

Theorem 8.4 (Inclusion preserves covers)**.** *Let $\iota\colon P \hookrightarrow P'$ be a program inclusion (adding modules or functions to P without modifying existing code). Then the induced functor $\iota^*\colon \mathcal{C}_{P'} \to \mathcal{C}_P$ preserves covering sieves: if $S \in \mathscr{J}_{P'}(c)$ and c is in the image of ι, then $(\iota^*)^{-1}(S) \in \mathscr{J}_P(\iota^*(c))$.*

Proof sketch. Since ι adds code without modifying existing coordinates, every covering family of a coordinate in P remains a covering family in P'. The functor ι^* is the identity on coordinates belonging to P, so preservation is immediate. For coordinates in $P' \setminus P$, the covering families are new and do not affect the topology restricted to P. □

Corollary 8.5 (Monotonicity of verification)**.** *If a program P is verified (i.e., the VC sheaf $\mathcal{V}$ admits a global section over $\mathcal{S}_P$) and P is extended to P' by adding new modules, then the restriction of the global section to the original coordinates remains valid.*

Proposition 8.6 (Sub-canonicity)**.** *The topology $\mathscr{J}_P$ is sub-canonical: every representable presheaf* $\mathrm{Hom}(-, c)$ *is a sheaf.*

Proof sketch. We must show that for every covering sieve $S \in \mathscr{J}_P(c)$ and every compatible family of morphisms into c, there is a unique amalgamation. Since $\mathcal{C}_P$ is a forest-based category, the covering families are *effective-epimorphic*: the coproduct $\coprod_i d_i \to c$ is an effective epimorphism in $\mathcal{C}_P$ (the "pieces" jointly determine any morphism into c). This is the defining property of sub-canonicity. □

9 Canonicity and Universality

A natural concern is whether our choice of site is *ad hoc*: might there be many incomparable Grothendieck topologies on $\mathcal{C}_P$, each yielding different verification outcomes? We show that the topology $\mathscr{J}_P$ is, in a precise sense, the *canonical* choice.

Definition 9.1 (Faithful topology)**.** A Grothendieck topology $\mathscr{J}$ on $\mathcal{C}_P$ is *faithful* if every covering sieve $S \in \mathscr{J}(c)$ is *semantically effective*: for every presheaf F of runtime behaviors and every compatible family $(s_i)_i$ on S, the glued section $s \in F(c)$ agrees with the actual behavior of the program at c.

Theorem 9.2 (Canonicity)**.** *Among all faithful, sub-canonical Grothendieck topologies on $\mathcal{C}_P$, the topology $\mathscr{J}_P$ is the* finest *(i.e., has the most covering sieves). Equivalently, $\mathscr{J}_P$ is the supremum in the lattice of faithful topologies.*

Proof. Let $\mathscr{K}$ be any faithful sub-canonical topology on $\mathcal{C}_P$. We show $\mathscr{K}(c) \subseteq \mathscr{J}_P(c)$ for every coordinate c.

Let $S \in \mathscr{K}(c)$ be generated by a family $\{r_i : d_i \to c\}$. Since $\mathscr{K}$ is faithful, the family is semantically effective: the d_i jointly determine the behavior at c. By the construction of $\mathscr{J}_P$, every semantically effective family is a covering family: control-flow covers capture branching structure, scope covers capture variable access, module covers capture imports, and temporal covers capture exception flow. The key observation is that *any* family that jointly determines the behavior at c must, by the structure of Python's execution model, factor through one of these four canonical cover types (branching, scoping, importing, or temporal ordering are the *only* ways that sub-regions of a Python program jointly determine the behavior of a larger region). Therefore $S \in \mathscr{J}_P(c)$.

Conversely, $\mathscr{J}_P$ is faithful by construction (Theorem 4.8) and sub-canonical (Proposition 8.6). □

Remark 9.3 (Connection to Lawvere's functorial semantics). F. W. Lawvere's programme of *functorial semantics* [9] interprets algebraic theories as product-preserving functors from a syntactic category to **Set**. Our construction extends this vision to verification: the site $\mathcal{S}_P$ is a *syntactic* category (built from the program's structure), and a sheaf on $\mathcal{S}_P$ is a *semantic* interpretation that respects the covering structure. The sheaf condition replaces Lawvere's product preservation with the richer requirement of *descent*, enabling the local-to-global reasoning that product-preserving functors alone cannot express. In this sense, judgment geometry is to Lawvere's programme what Grothendieck's scheme theory was to classical algebraic geometry: a passage from *affine* (global) to *general* (local-to-global) reasoning.

Theorem 9.4 (Complexity bound)**.** *For a Python program P with n AST nodes, f functions, and call-graph diameter d:*

(i) $|\mathrm{Ob}(\mathcal{C}_P)| = \Theta(n)$,
(ii) $|\mathrm{Mor}(\mathcal{C}_P)| = O(n \cdot d)$,
(iii) $|\mathrm{Cov}(\mathcal{C}_P)| = O(f)$,
(iv) the site $\mathcal{S}_P$ can be constructed in $O(n \log n)$ time.

In contrast, abstract-interpretation lattices over a domain with k abstract values have height $\Omega(k^f)$ (exponential in the number of functions), and Hoare-logic VCs for f mutually recursive functions require $\Omega(f^2)$ frame conditions.

Proof. (i) Each AST node generates at most one coordinate (function definitions, class definitions, branch points, and module roots generate coordinates; expression nodes do not). The constant factor is < 1 since only structurally significant nodes become coordinates.

(ii) Each coordinate has at most one restriction morphism to its parent and at most d transport morphisms (one per call-graph path). The total is bounded by $n \cdot (1 + d)$.

(iii) Each function body generates one covering family (its branch structure); modules generate one covering family over their top-level definitions. The total is $O(f)$.

(iv) AST parsing is $O(n)$; coordinate extraction is a single AST walk $O(n)$; morphism construction requires sorting children per scope $O(n \log n)$; covering family construction is $O(f)$. The dominant term is $O(n \log n)$.

For abstract interpretation, a non-relational domain with k values and f functions has a lattice of height k^f (the product lattice); the Kleene iteration may visit each lattice element. For Hoare logic with f mutually recursive functions, each function's frame condition must account for all others, yielding $\Omega(f^2)$ VCs. □

Corollary 9.5 (Polynomial verification overhead)**.** *The sheaf-theoretic verification pipeline adds at most $O(n \log n)$ overhead to program analysis, compared to $O(k^f)$ for abstract interpretation and $O(f^2)$ for Hoare-logic VC generation with mutual recursion.*

Theorem 9.6 (Embedding of existing frameworks)**.** *The following verification frameworks embed as special cases of $\mathcal{S}_P$:*

(i) ***Hoare logic****: the two-point site* $\{\mathsf{pre}, \mathsf{post}\}$ *with the unique non-trivial cover* $\{\mathsf{pre}, \mathsf{post}\} \twoheadrightarrow \mathsf{stmt}$ *is a full subcategory of $\mathcal{S}_P$ restricted to any single statement.*
(ii) ***Separation logic****: the heap-partition site, where coordinates are heap regions and covers are spatial decompositions (the separating conjunction $P * Q$ is a covering family $\{P, Q\} \twoheadrightarrow P * Q$), embeds via the functor mapping heap regions to scope coordinates.*
(iii) ***Abstract interpretation****: a Galois connection (α, γ) between concrete and abstract domains defines a two-object presheaf on the restriction* $\mathsf{abstract} \to \mathsf{concrete}$*; the soundness condition $\alpha \circ f \leq f^\sharp \circ \alpha$ is the sheaf condition.*

Proof sketch. Each embedding is a fully faithful functor from the respective verification category into $\mathcal{C}_P$. For (i), the functor maps pre to the function-entry coordinate and post to the function-exit coordinate. For (ii), the frame rule of separation logic ($\{P\}\,C\,\{Q\} \implies \{P * R\}\,C\,\{Q * R\}$) corresponds exactly to the stability axiom (T2) of the Grothendieck topology: pulling back the cover $\{P, Q\}$ along the inclusion $P * R \hookrightarrow (P * Q) * R$ yields a cover. For (iii), the Galois connection defines a morphism of presheaves; soundness is the naturality condition. □

10 Related Work

Sheaves in computer science. Sheaf theory has appeared in computer science primarily through the lens of *sheaves on topological spaces* for concurrent and distributed systems [7]. Abramsky and colleagues [1] used sheaves to model non-locality and contextuality in quantum mechanics, an approach later adapted to databases and constraint satisfaction. Our contribution is to apply *Grothendieck sites* (rather than topological spaces) directly to program structure, where the covering families arise from the program's own decomposition rather than an externally imposed topology.

Abstract interpretation. Cousot and Cousot's framework [4] can be viewed as a special case of our construction. An abstract domain $D^\sharp$ with concretization $\gamma\colon D^\sharp \to \wp(D)$ defines a presheaf on the two-object site $\{\mathsf{abstract}, \mathsf{concrete}\}$ with a single non-trivial cover $\{\mathsf{abstract} \to \mathsf{concrete}\}$. The soundness condition $\gamma \circ f^\sharp \supseteq f \circ \gamma$ is precisely the sheaf condition for this presheaf. Our framework generalizes this by allowing an arbitrary number of abstraction levels (one per coordinate) with arbitrary covering structure.

Hoare logic and separation logic. Hoare triples $\{P\}\ C\ \{Q\}$ can be modeled as sections of a presheaf over a two-point site (pre-state, post-state). O'Hearn's separation logic [12] enriches this with a spatial conjunction $*$ that decomposes the heap into disjoint regions—a special case of our scope covers where the overlap is guaranteed to be empty. The *frame rule* of separation logic, which states that properties of disjoint heap regions compose freely, is an instance of sheaf descent for a cover with trivial overlaps.

Dependent type systems. Lean 4 [5] and F* [14] use dependent types to express specifications and extract verified code. The Curry–Howard correspondence gives these systems a built-in local-to-global mechanism (type checking is compositional), but the "topology" is implicit in the type system's scoping rules. Our framework makes this topology explicit and extends it beyond the scope structure to include control flow and temporal decomposition.

Voevodsky's Homotopy Type Theory [15, 8] demonstrates that type-theoretic universes carry non-trivial homotopical structure. Our semantic sites can be seen as a *concrete* incarnation of this idea: the coordinates form a space, the covering families define its topology, and sheaf descent replaces the univalence axiom as the mechanism for transporting proofs across equivalent regions.

Modular verification and compositional reasoning. The principle that programs should be verified modularly is well established in the verification literature. Cardelli's work on type systems for modular programs [3] provides a type-theoretic foundation. Calcagno et al. [2] develop *compositional shape analysis* using bi-abduction, where local heap specifications are composed into global ones—an instance of our gluing principle specialized to separation-logic assertions over heap shapes.

The key difference between our approach and prior compositional verification is *generality*: where each prior system (separation logic, rely-guarantee, assume-guarantee) develops its own composition principle, we provide a single framework—sheaf descent—that subsumes them all as special cases corresponding to particular choices of covering families and presheaves.

Program verification tools. DAFNY [10] compiles method contracts to Boogie verification conditions, achieving modular reasoning via method-level specifications. Why3 [6] provides a similar architecture with backend-agnostic verification conditions. These tools verify programs module-by-module but lack a mathematical characterization of *when* and *why* modular reasoning is sound. Our sheaf-descent theorem provides exactly this characterization.

11 Conclusion

We have introduced *semantic sites*—categories of program coordinates equipped with Grothendieck topologies—as a mathematical foundation for program verification. The key insight is that a program's decomposition into modules, functions, branches, and loop bodies is not merely a syntactic convenience but a *topological structure* over which verification conditions form sheaves. The sheaf condition captures exactly when local guarantees glue into global ones, and the descent theorem provides a constructive procedure for performing this gluing.

The semantic site $\mathcal{S}_P$ for a Python program P satisfies three structural properties—enough points, local connectedness, and cover-preservation under inclusion—that underpin the remainder of the Judgment Geometry series. Empirically, the framework demonstrates linear scaling of site complexity on a benchmark of 18 programs across four size categories, with negligible construction overhead (0.05 ms per program).

Future work. Paper 2 of the series introduces the *judgment 8-tuple* $\mathcal{J} = (\mathsf{c}, \varphi, \mathcal{A}, \mathcal{E}, \mathcal{O}, \mathcal{B}, \mathcal{T}, \Pi)$, which extends the sheaf-section formalism with evidence bundles, trust grading, and obstruction tracking. Paper 3 develops the Čech cohomology of $\mathcal{S}_P$ and uses it to detect and repair *obstructions* to gluing— cases where local specifications *fail* to extend globally. Together, these three papers provide the theoretical core of sheaf-theoretic program verification.

The full Judgment Geometry programme comprises ten papers:

1. Semantic sites (this paper);
2. The judgment 8-tuple algebra;
3. Sheaf descent and Čech obstructions;
4. Trust-ordered algebra for mixed-evidence verification;
5. Fragment-aware SMT dispatch;
6. Semantic moves (tactics over proof geometry);
7. Python runtime effects as typed sheaf sections;
8. Treaty synthesis in hypercover families;
9. Proof-carrying Python via semantic scaffolds;
10. Empirical evaluation on 300 programs.

A seminal paper synthesizes the entire programme. Each paper is designed to be self-contained; dependencies between papers are indicated by forward references.

Reproducibility. All code, benchmarks, and evaluation scripts are available in the JUGEO repository. The benchmark suite (18 programs across four size categories) runs in under two minutes on a single core.

References

[1] S. Abramsky and A. Brandenburger. The sheaf-theoretic structure of non-locality and contextuality. *New Journal of Physics*, 13(11):113036, 2011.

[2] C. Calcagno, D. Distefano, P. W. O'Hearn, and H. Yang. Compositional shape analysis by means of bi-abduction. In *Proc. POPL*, pages 289–300. ACM, 2007.

[3] L. Cardelli. Program fragments, linking, and modularization. In *Proc. POPL*, pages 266–277. ACM, 1997.

[4] P. Cousot and R. Cousot. Abstract interpretation: a unified lattice model for static analysis of programs by construction or approximation of fixpoints. In *Proc. POPL*, pages 238–252. ACM, 1977.

[5] L. de Moura, S. Kong, J. Avigad, F. van Doorn, and J. von Raumer. The Lean 4 theorem prover and programming language. In *Proc. CADE*, volume 12699 of *LNCS*, pages 625–635. Springer, 2021.

[6] J.-C. Filliâtre and A. Paskevich. Why3 — where programs meet provers. In *Proc. ESOP*, volume 7792 of *LNCS*, pages 125–128. Springer, 2013.

[7] J. A. Goguen. Sheaf semantics for concurrent interacting objects. *Mathematical Structures in Computer Science*, 2(2):159–191, 1992.

[8] The Univalent Foundations Program. *Homotopy Type Theory: Univalent Foundations of Mathematics*. Institute for Advanced Study, 2013.

[9] F. W. Lawvere. Functorial semantics of algebraic theories. *Proceedings of the National Academy of Sciences*, 50(5):869–872, 1963.

[10] K. R. M. Leino. Dafny: An automatic program verifier for functional correctness. In *Proc. LPAR*, volume 6355 of *LNCS*, pages 348–370. Springer, 2010.

[11] S. Mac Lane and I. Moerdijk. *Sheaves in Geometry and Logic: A First Introduction to Topos Theory*. Universitext. Springer, corrected edition, 1994.

[12] P. W. O'Hearn. Separation logic. *Communications of the ACM*, 62(2):86–95, 2019.

[13] M. Artin, A. Grothendieck, and J.-L. Verdier. *Théorie des Topos et Cohomologie Étale des Schémas (SGA 4)*. Lecture Notes in Mathematics 269, 270, 305. Springer, 1972–1973.

[14] N. Swamy, C. Hriţcu, C. Keller, A. Rastogi, A. Delignat-Lavaud, S. Forest, K. Bhargavan, C. Fournet, P.-Y. Strub, M. Kohlweiss, J.-K. Zinzindohoué, and S. Zanella-Béguelin. Dependent types and multi-monadic effects in F*. In *Proc. POPL*, pages 256–270. ACM, 2016.

[15] V. Voevodsky. A univalent formalization of the p-adic numbers. *Mathematical Structures in Computer Science*, 25(5):1147–1171, 2015.

A Proof Details for Theorem 4.8

We provide additional detail for the stability axiom (T2).

Lemma A.1 (Pullback of control-flow covers)**.** *Let $S = \mathrm{Cov}_{\mathrm{cf}}(c) = \{r_i \colon b_i \to c\}$ be a control-flow cover and $h \colon e \to c$ a morphism. Then:*

(a) If h is a restriction and e is a proper ancestor of c, then $h^*(S) = t_e$ (the maximal sieve).
(b) If h is a restriction and $e = b_j$ for some j, then $h^*(S) = t_{b_j}$.
(c) If h is a transport and e is a sibling of c, then $h^*(S)$ is the control-flow cover of e induced by the corresponding branches in e (if they exist), or the maximal sieve (if e has no branching).
(d) If h is a refinement, then $h^*(S)$ is the refinement of the covering family, which is a covering sieve by transitivity.

Proof. Cases (a) and (b) follow directly from the definition of pullback sieve: $h^*(S) = \{g \mid h \circ g \in S\}$. When e is an ancestor, every morphism to e composes with h to land in S (since S is closed under precomposition and the maximal sieve contains h). When $e = b_j$, the identity on b_j is the unique morphism such that $r_j \circ \mathrm{id} = r_j \in S$.

Case (c) uses the fact that transport morphisms preserve branching structure (Definition 3.2(c)), so the corresponding branches in e map to the branches in c under the transport.

Case (d) is immediate from transitivity (T3): a refinement followed by a cover is a cover. □

B Categorical Diagram: the Descent Condition

The sheaf (descent) condition for a covering family $\{U_i \to X\}$ can be expressed as the exactness of the following equalizer diagram:

$$F(X) \xrightarrow{\prod \mathrm{res}_{U_i}} \prod_i F(U_i) \underset{\pi_2}{\overset{\pi_1}{\rightrightarrows}} \prod_{i,j} F(U_i \times_X U_j)$$

Here $F(X) \to \prod_i F(U_i)$ is the product of restriction maps, and π_1, π_2 are the two projections from each $F(U_i)$ and $F(U_j)$ to the overlap $F(U_i \times_X U_j)$. The equalizer condition states that $F(X)$ is the set of compatible families—exactly the gluing axiom.

For program verification, X is a function scope, the U_i are its branches, and $U_i \times_X U_j$ is the overlap (shared variables and state). A global specification $s \in F(X)$ is uniquely determined by its restrictions to the branches, provided they agree on overlaps.

C Glossary of Notation

For the reader's convenience, we collect the principal notation used throughout the paper.

Symbol	Meaning
$\mathcal{C}_P$	Category of program coordinates for program P
$\mathscr{J}_P$	Grothendieck topology on $\mathcal{C}_P$
$\mathcal{S}_P$	Semantic site ($\mathcal{C}_P$, $\mathscr{J}_P$)
$\mathrm{Cov}(c)$	Covering family of coordinate c
res	Restriction morphism (scope narrowing)
$\mathcal{T}$	Presheaf of types
$\mathcal{V}$	Presheaf of verification conditions
$\mathcal{R}$	Presheaf of test outcomes
$\widehat{\mathcal{C}}$	Category of presheaves on $\mathcal{C}$
$\mathbf{Sh}(\mathcal{S}_P)$	Category of sheaves on $\mathcal{S}_P$
RUNTIME	Trust level: runtime-witnessed
SOLVER	Trust level: SMT-solver-verified
PROOF	Trust level: symbolically proved
$h^*(S)$	Pullback sieve of S along h
t_c	Maximal sieve on c